```
void setup() {
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}
```

Starting at the beginning, we specify the function's data type, in this case `void`, which is not terribly important for the moment, followed by the function's name, `setup`, an opening and closing parenthesis, `( )`, and a block of code bounded on either side with opening and closing curly braces, `{ }`. The three lines of code inside the curly braces are examples of statements used to set up the state of the digital pins on the Arduino interface board. Specifically, we're telling the Arduino that pins 9, 10, and 11 should be used as outputs and that we intend to use them a little later on in the sketch.

Of these two functions, the `setup()` function is called only once at the beginning of our sketch and is used for setting up various aspects of the Arduino board for running our code. A **function call** is another way of saying that the function, and all of the functions contained within it, is executed in the order they are listed. Examples of using the `setup()` function might also include establishing serial communications, making initial sensor readings, or running certain tasks that are only needed one time. This function will always run once, immediately after powering up or resetting the Arduino board.

■ **Note** Anytime we discuss a function in this book, we will use a pair of parenthesis after its name to help distinguish functions from things like variables. As an example, `assignValue` refers to a variable (more on these in a moment) where `assignValue()` refers to a function call.

The second function, `loop()`, makes it possible for your sketch to run continuously, and begins after the last statement in the `setup()` function has finished executing. Once the Arduino has completed its statements in order, reaching the end of the statements in the `loop()` function, it will, appropriately enough, loop back to the top of the `loop()` function and begin again, ad infinitum. Chances are this is where the bulk of your Arduino sketches will reside. Later in this book, we will investigate ways to manipulate the flow of the program and even demonstrate some methods for bouncing information back and forth between functions of your own creation.

■ **Note** When the Arduino runs a sketch, it starts at the top and works its way towards the bottom. This is called **program flow** and refers to the order that the statements are executed. Much like the way the `loop()` function continuously repeats when it reaches the end of its statements, there are other ways to modify the program flow, which will be discussed later.

Statements and Syntax

As mentioned earlier, nestled inside the curly braces of functions are lines of code called **statements**. Quite simply, these are the parts of the sketch that actually do something, such as calls to other functions, including those functions that are a part of the standard Arduino library, declaring and